SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 기본

List 1

Python

학습시작

내가 좋아하는 음식 리스트를 만들어 봅시다!

1. 변수 [ ] 에 내가 좋아하는 음식들을 리스트 형태로 저장합니다
2. 리스트에 몇 개의 음식이 들어 있는지 확인하기 위해 [ ] 을 사용해 봅니다
3. 리스트에서 세 번째 음식을 꺼내기 위해 [ ] 형태로 특정 값을 출력합니다

- 여러 값을 하나의 묶음으로 저장할 수 있는 자료형인 리스트(list)를 배울 수 있어요
- 리스트를 사용하면 하나의 선언을 통해서 둘 이상의 변수를 선언할 수 있어요
- 리스트에 저장된 값 중 특정 위치(index)의 값을 꺼내 활용하는 방법도 익힐 수 있어요

Objectives

학습목표

- 알고리즘의 개념을 이해하고 설명할 수 있다
- 리스트의 개념과 필요성을 이해하고 설명할 수 있다
- 1차원 리스트를 활용할 수 있다
- 정렬의 개념에 대하여 설명할 수 있다
- 버블 정렬의 정렬 방식을 설명할 수 있다
- 카운팅 정렬의 정렬 방식을 설명할 수 있다
- 완전 검색의 개념을 설명할 수 있다
- 탐욕 알고리즘에 대해 설명할 수 있다

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

List

- 과정 소개
- 알고리즘
- 배열
- 연습 문제
- 버블 정렬 (Bubble Sort)
- 카운팅 정렬 (Counting Sort)
- 완전 검색
- 탐욕 알고리즘

List

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

과정 소개

학습 내용

- 자료구조와 알고리즘

학습 목표

- 논리적 사고력 향상
- 문제 해결 능력 향상

⇨ 최종 목표는 연습 문제가 아닌 “현실 세계 문제”

학습 도구

- Python (Pypy) + PyCharm + swexpertacademy.com(SWEA) + AI

APS(Algorithm Problem Solving) 기본 학습

- 기초 자료 구조 이해 중심
  - 입출력을 제외한 내장함수를 사용하지 않습니다.
  - 기본적인 내장함수의 동작 원리를 이해합니다.

- AI 활용 방법
  - 교과서가 아닌 학습에 도움을 받는 용도로 사용합니다.
  - 문제 전체를 넘기고 코드를 요구해서는 안됩니다.
  - 대화를 통해 문제점을 찾도록 작은 단위로 질문해봅시다.

TIP

- 다른 사람이 작성한 코드를 많이 읽어보는 것이 중요합니다.
- 코드를 눈으로 보는 것이 아니라 직접 따라 입력해 보세요.
- 코드가 외워질 정도로 반복해야 이해도 잘 할 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

알고리즘

알고리즘이란?

알고리즘(Algorithm)

문제를 해결하기 위한
절차나 방법입니다.

주로 컴퓨터 용어로 쓰이며, 컴퓨터가 어떤 일을 수행하기 위한 단계적 방법을 말합니다.

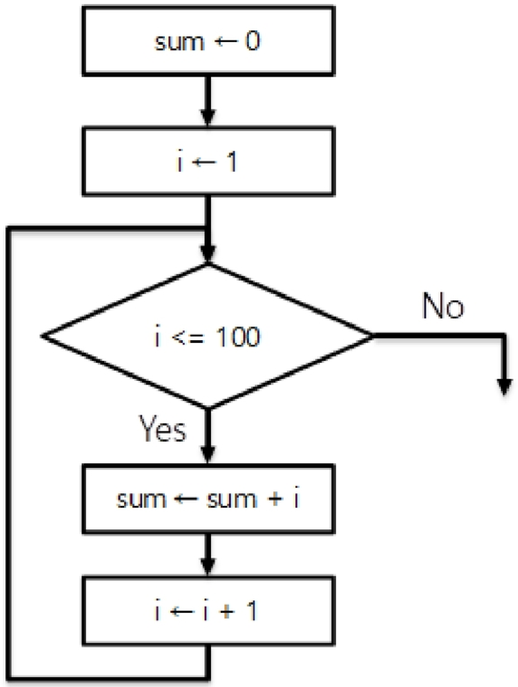
간단하게 다시 말하면, 어떠한 문제를 해결하기 위한 절차라고 볼 수 있습니다.

의사코드(슈도코드, Pseudocode)와 순서도

- 컴퓨터 분야에서 알고리즘을 표현하는 방법

```
CalcSum( n )
    sum <- 0
  for i : 1 -> n
     sum <- sum + i
  return sum
```

<그림1_SW문제해결 기본_List1_의사코드 예시>



<그림2 _SW문제해결 기본_List1_ 순서도 예시>

알고리즘의 성능

- APS 과정의 목표 중 하나 : 보다 좋은 알고리즘을 이해하고 활용하기
- 좋은 알고리즘이란?

정확성	얼마나 정확하게 동작하는가
작업량	얼마나 적은 연산으로 원하는 결과를 얻어내는가
메모리 사용량	얼마나 적은 메모리를 사용하는가
단순성	얼마나 단순한가
최적성	더 이상 개선할 여지없이 최적화되었는가

알고리즘의 성능

- 주어진 문제를 해결하기 위해 여러 개의 다양한 알고리즘이 가능
  - 어떤 알고리즘을 사용해야 하는가를 고려해야 합니다.

- 알고리즘의 성능 분석 필요
  - 많은 문제에서 성능 분석의 기준으로 알고리즘의 작업량을 비교합니다.

알고리즘 성능 비교 예시

- 1부터 100까지 합을 구하는 문제

알고리즘 1	알고리즘 2
1 + 2 + 3 + 4 ..... + 100 5,050	$\frac{100 \times (1+100)}{2} = 5,050$
100번의 연산 (덧셈 100번)	3번의 연산 (덧셈 1번, 곱셈 1번, 나눗셈 1번)

<표1_SW문제해결 기본_List1_1부터 100까지 합 구하기>

알고리즘의 시간 복잡도

- 알고리즘의 작업량을 표현할 때 시간 복잡도로 표현합니다.
- 시간 복잡도(Time Complexity)
  - 실제 걸리는 시간을 측정
  - 실행되는 명령문의 개수를 계산

알고리즘 성능 비교 예시

- 1부터 n까지 합을 구하는 문제

알고리즘 1	알고리즘 2

def CalcSum(n) :
  sum ← 0 # 1번
  for i in range (1, n+1) : # n번
  sum ← sum + i # n번
  return sum
	def CalcSum(n) :
	  return n*(n+1)//2 #3번

1 + n * 2 = 2n + 1	3번의 연산

<표2_SW문제해결 기본_List1_1부터 n까지 합 구하기>

시간 복잡도 표시

- 빅-오 표기법(Big-O Notation)을 언급하는 경우가 많음
- 시간 복잡도 함수 중에서 가장 큰 영향력을 주는 n에 대한 항만을 표시
- 계수(Coefficient)는 생략하여 표시

$$O(3n + 2) = O(3n) = O(n)$$

최고차항(3n)만 선택 계수 3 제거

$$O(2n^2 + 10n + 100) = O(n^2)$$

$$O(4) = O(1)$$

빅-오(O) 표기법 적용 예

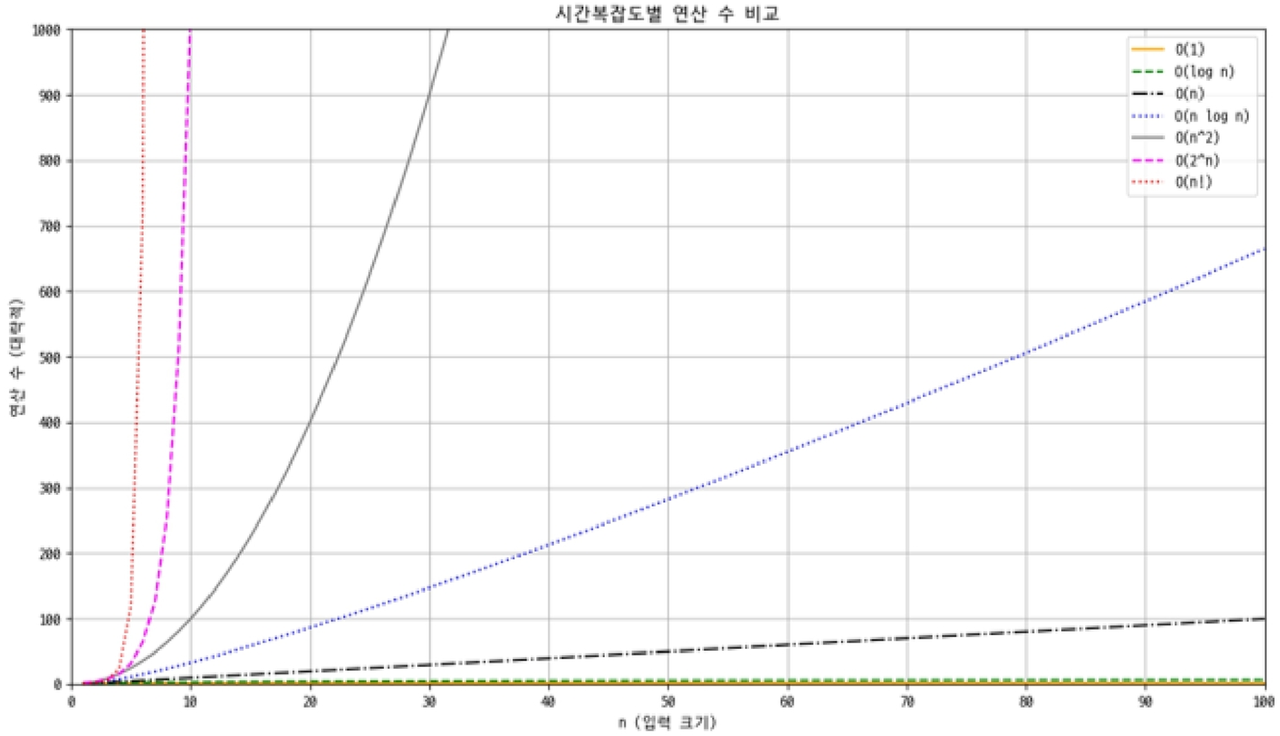
n개의 데이터를 입력받아 저장한 후 각 데이터에 1씩 증가시킨 후
각 데이터를 화면에 출력하는 알고리즘의 시간 복잡도는?

※ 정답 : O(n)

빅-오 표기법(Big-O Notation)

- 비교

- 요소 수가 증가함에 따라 각기 다른 시간 복잡도의 알고리즘은 아래와 같은 연산 수를 보입니다.



<그림3_SW문제해결 기본_List1_Big O 시간 복잡도>

시간 복잡도별 실제 실행 시간 비교

Q 5천만 번 연산을 수행한다면?

N	$\log N$	N	$N\log N$	N^2	N^3	2^N
10	0.003 μs	0.01 μs	0.033 μs	0.1 μs	1 μs	1 μs
20	0.004 μs	0.02 μs	0.086 μs	0.4 μs	8 μs	1 ms
30	0.005 μs	0.03 μs	0.147 μs	0.9 μs	27 μs	1 s
40	0.005 μs	0.04 μs	0.213 μs	1.6 μs	64 μs	18.3 min
50	0.006 μs	0.05 μs	0.282 μs	2.5 μs	125 μs	13 days
10^2	0.007 μs	0.10 μs	0.664 μs	10 μs	1 ms	4 X 10^{13} years
10^3	0.010 μs	1.00 μs	9.966 μs	1 ms	1 s	
10^4	0.013 μs	10 μs	130 μs	100 ms	16.7 min	
10^5	0.017 μs	0.10 ms	1.67 ms	10 s	11.6 days	
10^6	0.020 μs	1 ms	19.93 ms	16.7 min	31.7 days	
10^7	0.023 μs	0.01 s	0.23 s	1.16 days	31,709 years	
1 억 10^8	0.027 μs	0.10 s	2.66 s	115.7 days	3.17 X 10^7 years	
10^9	0.030 μs	1 s	29.90 s	31.7 years		

1μs = 10^{-6}초
1ms = 10^{-3}초

<표3_SW문제해결 기본_List1_시간 복잡도별 실제 실행 시간 비교>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

배열

배열의 정의

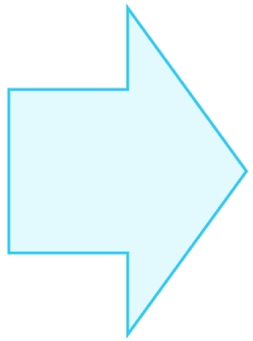
배열
Array

일정한 자료형의 변수들을
하나의 이름으로 열거하여 사용하는 자료구조

배열의 예시

- 6개의 변수를 사용해야 하는 경우
  - 이를 배열로 바꾸어 사용하는 예시입니다

```
num0 = 0;
num1 = 1;
num2 = 2;
num3 = 3;
num4 = 4;
num5 = 5;
```



```
num = [0,1,2,3,4,5]
```

배열의 필요성

- 프로그램 내에서 여러 개의 변수가 필요할 때

  - 일일이 다른 변수명을 이용하여 자료에 접근하는 것은 매우 비효율적일 수 있습니다.

- 배열을 사용하면 하나의 선언을 통해서 둘 이상의 변수를 선언
- 단순히 다수의 변수 선언을 의미하는 것이 아님

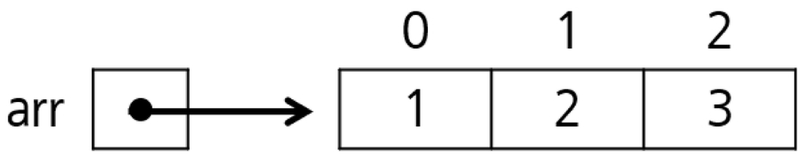
  - 다수의 변수로는 하기 힘든 작업을 배열을 활용해 쉽게 할 수 있습니다.

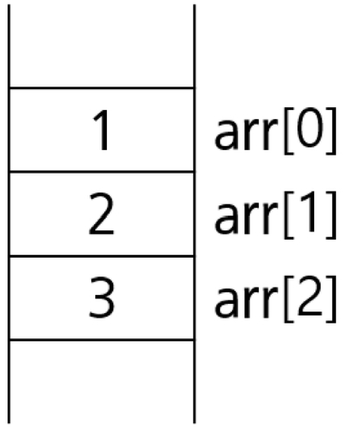
1차원 배열

- 배열을 선언할 때는 프로그램에서 사용할 배열 이름이 필요합니다.

1차원 배열 선언의 예시입니다

```
arr = list()
arr = [ ]
arr = [0] * 10
arr = [1,2,3]
```





<그림4_SW문제해결 기본_List1_1차원 배열 선언 예시>

1차원 배열 접근의 예시입니다

```
arr[0] = 10   # '배열 arr의 0번 원소에 10을 저장하라'
arr[idx] = 20 # '배열 arr의 idx번 원소에 20을 저장하라'
```

입력 받은 정수를 1차원 배열에 저장하는 방법

- 첫 줄에 양수의 개수 N이 주어집니다. (5 ≤ N ≤ 1000)
- 다음 줄에 빈칸으로 구분된 N개의 양수 Ai가 주어집니다. (1 ≤ Ai≤ 1000000)

입력 예시입니다

```
6
2 7 5 3 1 4
```

인덱스	0	1	2	3	4	5	N=6
값	2	7	5	3	1	4	

<그림5_SW문제해결 기본_List1_1차원 배열 예시>

정수 N을 입력 받고, 공백으로 구분된 정수들을 리스트로 입력 받는 코드입니다

```
N = int(input())
arr = list(map(int, input().split()))
```

배열 원소의 합 s 계산하기

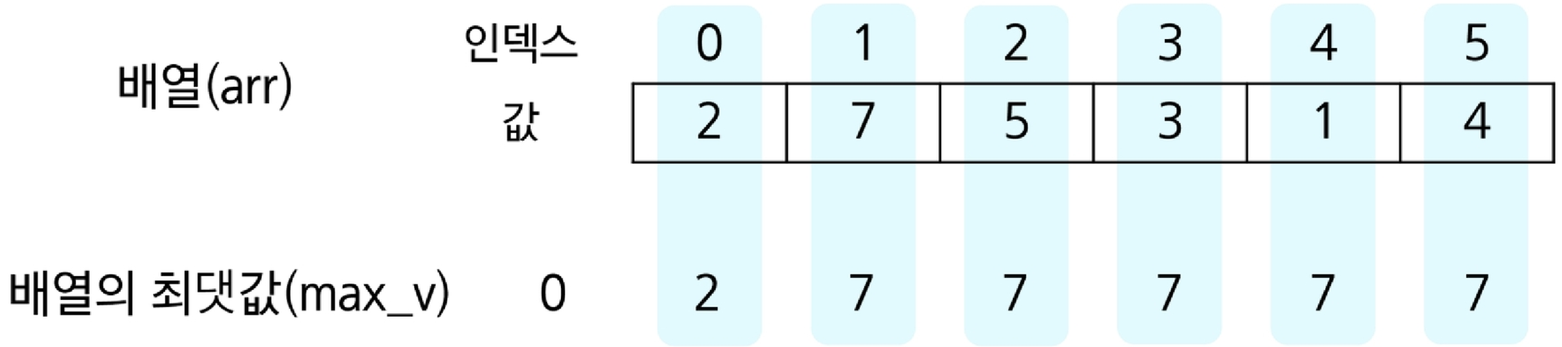
배열(arr)	인덱스	0	1	2	3	4	5
	값	2	7	5	3	1	4
배열의 합(s)	0	2	9	14	17	18	22

<그림6_SW문제해결 기본_List1_배열 원소의 합>

배열 원소의 합을 계산하는 코드입니다

```
s = 0
for i in range(N):      # for x in arr:
    s += arr[i]         #     s += x
```

배열 원소 중 최댓값 max_v 찾기

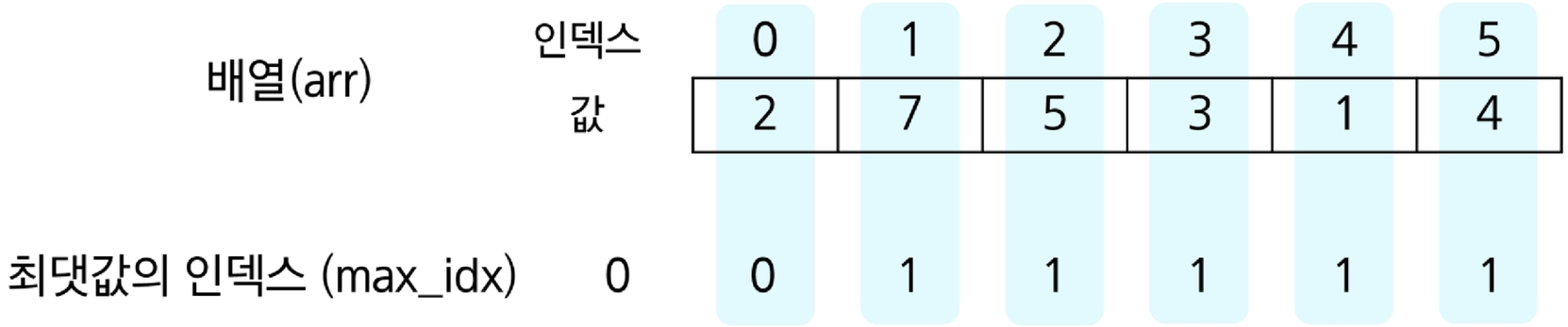


<그림7_SW문제해결 기본_List1_배열 원소 중 최댓값>

배열 원소의 최댓값을 찾는 코드입니다

```
max_v = arr[0]  # 첫 원소를 최대로 가정
for i in range(1, N):
    if max_v < arr[i]:
        max_v = arr[i]  # arr[i]가 더 크면 max_v 갱신
```

배열 원소 중 최댓값의 인덱스 max_idx 찾기



<그림8_SW문제해결 기본_List1_최댓값의 인덱스>

배열 원소의 최댓값의 인덱스를 찾는 코드입니다

```
max_idx = 0 # 첫 원소를 최대로 가정
for i in range(1, N):
    if arr[max_idx] < arr[i]:   # 더 큰값을 만나면
        max_idx = i             # max_idx 갱신
```

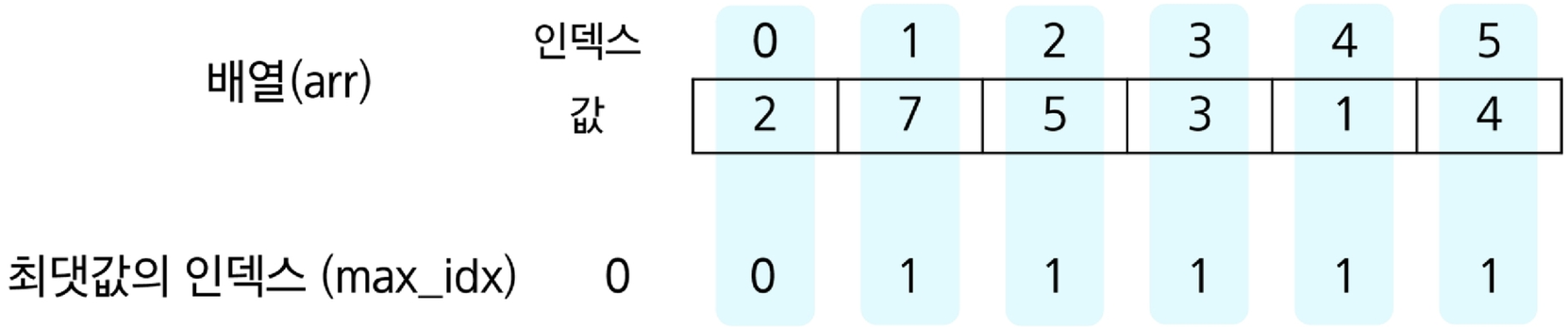
배열 원소 중 최댓값의 인덱스 max_idx 찾기

배열(arr)	인덱스	0	1	2	3	4	5
	값	2	7	5	3	1	4
최댓값의 인덱스 (max_idx)	0	0	1	1	1	1	1

<그림8_SW문제해결 기본_List1_최댓값의 인덱스>

Q 최댓값이 여러 개인 경우?

배열 원소 중 최댓값의 인덱스 max_idx 찾기



<그림8_SW문제해결 기본_List1_최댓값의 인덱스>

Q 최댓값이 여러 개인 경우?

가장 왼쪽의 최댓값 인덱스

최댓값이 여러 개인 경우 마지막 인덱스 max_idx 찾기

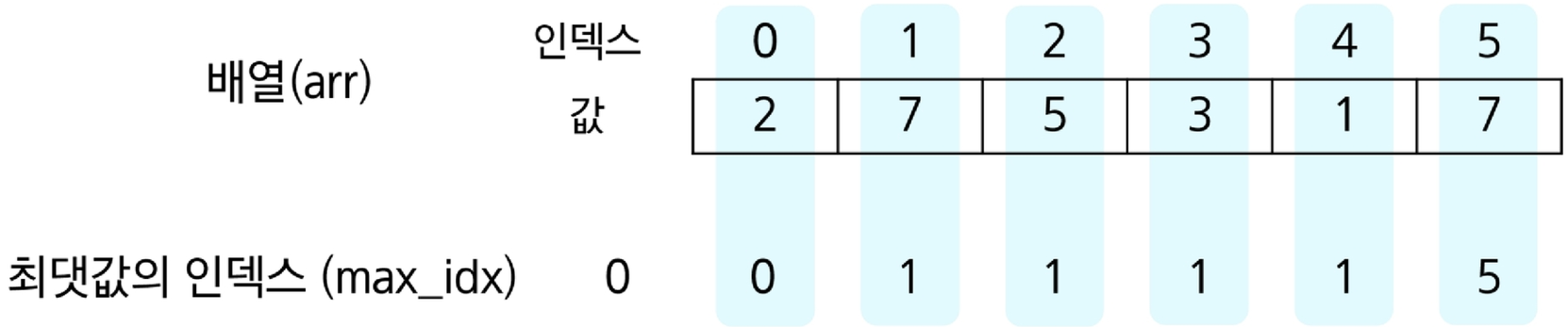
입력 예시입니다

```
6
2 7 5 3 1 7
```

배열(arr)	인덱스	0	1	2	3	4	5
	값	2	7	5	3	1	7

<그림9_SW문제해결 기본_List1_최댓값이 여러 개인 배열>

최댓값이 여러 개인 경우 마지막 인덱스 max_idx 찾기



<그림10_SW문제해결 기본_List1_최댓값이 여러 개인 경우 마지막 인덱스>

배열 원소의 최댓값의 마지막 인덱스를 찾는 코드입니다

```
max_idx = 0 # 첫 원소를 최대로 가정
for i in range(1, N):
    if arr[max_idx] <= arr[i]:  # 더 큰값 또는 같은값이면
        max_idx = i             # max_idx 갱신
```

찾는 값이 배열에 있으면 해당 원소의 인덱스, 없으면 -1을 idx에 넣기

입력 예시입니다

```
6 5
2 7 5 3 1 7
```

배열(arr)

인덱스	0	1	2	3	4	5
값	2	7	5	3	1	7

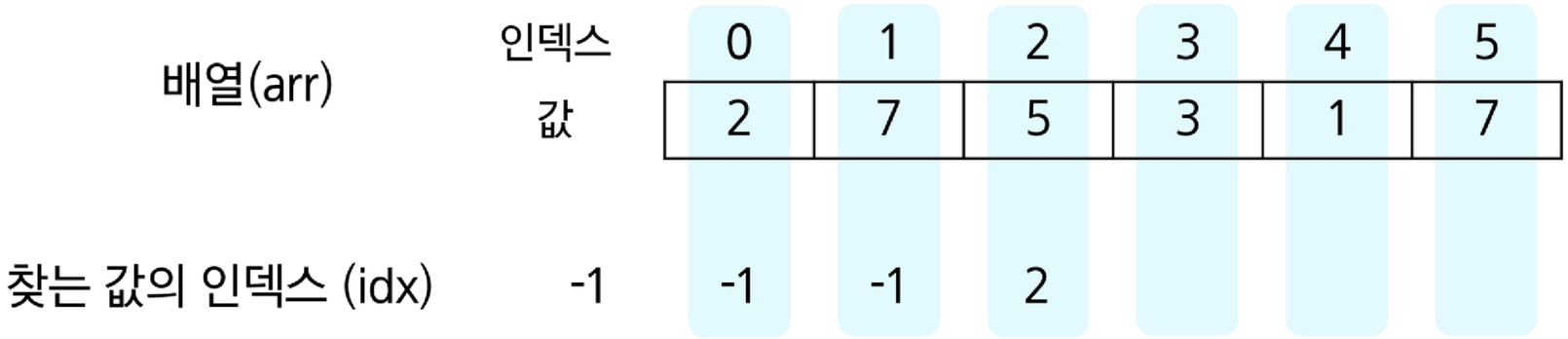
<그림9_SW문제해결 기본_List1_최댓값이 여러 개인 배열>

정수 N을 입력 받고, 공백으로 구분된 정수들을 리스트로 입력 받는 코드입니다

```
N, V = map(int, input().split()) # N, 찾는 값 V
arr = list(map(int, input().split()))
```

찾는 값이 배열에 있으면 해당 원소의 인덱스, 없으면 -1을 idx에 넣기

- 5를 찾는 경우



<그림11_SW문제해결 기본_List1_배열 원소의 검색>

배열에서 V값의 인덱스를 찾는 코드입니다

```
idx = -1              # 찾는 값이 없다고 가정
for i in range(N):
    if arr[i] == V: # arr[i]가 찾는 값이면
        idx = i     # 인덱스 저장
        break       # for i
```

연습 문제

연습 문제 1 (SWEA 4828)

- N개의 양의 정수에서 가장 큰 수와 가장 작은 수의 차이를 출력하세요.

입력

첫 줄에 테스트 케이스의 수 T가 주어진다. (1 ≤ T ≤ 50)
각 케이스의 첫 줄에 양수의 개수 N이 주어진다. (5 ≤ N ≤ 1000)
다음 줄에 N개의 양수 ai가 주어진다. (1 ≤ ai≤ 1000000)

출력

각 줄마다 "#T" (T는 테스트 케이스 번호)를 출력한 뒤, 빈칸에 이어 답을 출력한다.

연습 문제 1

입력 예시

```
3
5
477162 658880 751280 927930 297191
5
565469 851600 460874 148692 111090
10
784386 279993 982220 996285 614710 992232 195265 359810 919192 158175
```

출력 예시

```
#1 630739
#2 740510
#3 838110
```

연습 문제 2

- 배열 활용 예제 : Gravity

문제

상자들이 쌓여있는 방이 있다. 방이 오른쪽으로 90도 회전하여 상자들이 중력의 영향을 받아 낙하한다고 할 때, 낙차가 가장 큰 상자를 구하여 그 낙차를 리턴 하는 프로그램을 작성하시오.

- 중력은 회전이 완료된 후 적용된다.
- 상자들은 모두 한쪽 벽면에 붙여진 상태로 쌓여 2차원의 형태를 이루며 벽에서 떨어져서 쌓인 상자는 없다.
- 상자의 가로, 세로 길이는 각각 1이다.
- 방의 가로 길이는 최대 100이며, 세로 길이도 최대 100이다.
- 즉, 상자는 최소 0, 최대 100 높이로 쌓을 수 있다.
- 상자가 놓인 가로 칸의 수 N, 다음 줄에 각 칸의 상자 높이가 주어진다.

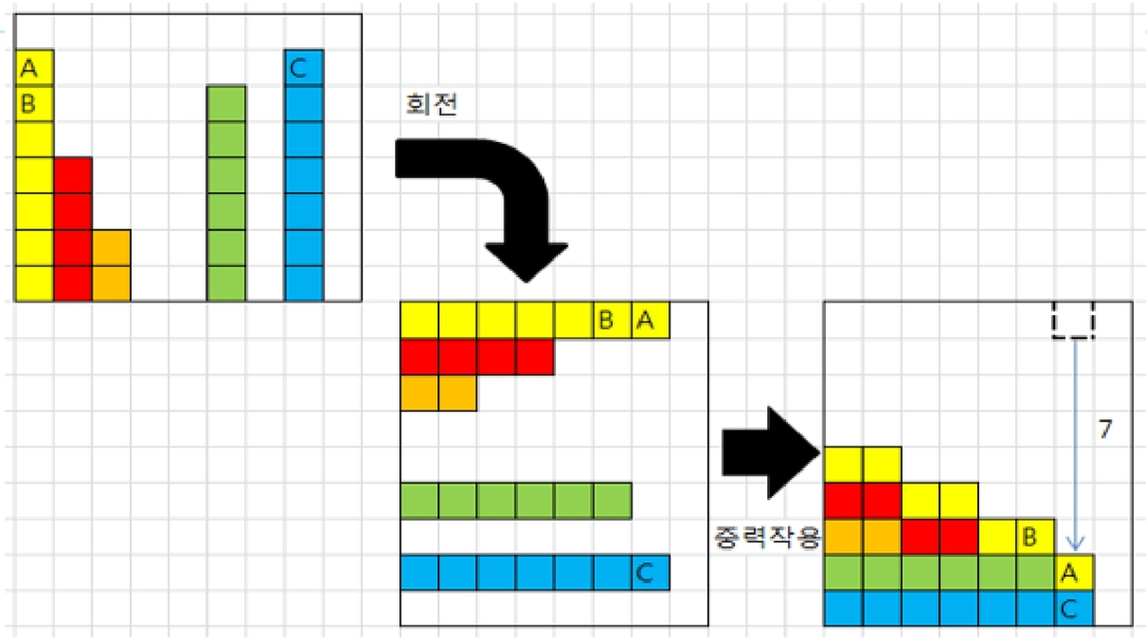
연습 문제 2

그림 설명

아래 예) 총 26개의 상자가 회전 후, 오른쪽 방 그림의 상태가 된다.
A 상자의 낙차가 7로 가장 크므로 7을 리턴하면 된다.
회전 결과, B상자의 낙차는 6, C상자의 낙차는 1이다.

입력 예시

```
9
7 4 2 0 0 6 0 7 0
```



<그림12_SW문제해결 기본_List1_낙차 문제 예시>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

버블 정렬

정렬의 정의

키 :
자료를 정렬하는 기준이 되는 특정 값

<개념1_SW문제해결 기본_List1_키>

정렬
Sort

2개 이상의 자료를 키(특정 기준)에 의해
작은 값부터 큰 값(오름차순 : ascending),
혹은 그 반대의 순서대로(내림차순 : descending) 재배열하는 알고리즘

정렬의 종류

- APS 과정을 통해 자료구조와 알고리즘을 학습하면서 다양한 형태의 정렬을 학습
- 대표적인 정렬 방식의 종류

버블 정렬 (Bubble Sort)

퀵 정렬 (Quick Sort)

카운팅 정렬 (Counting Sort)

삽입 정렬 (Insertion Sort)

선택 정렬 (Selection Sort)

병합 정렬 (Merge Sort)

버블 정렬의 정의

버블 정렬
Bubble Sort

인접한 두 개의 원소를 비교하며
자리를 계속 교환하는 방식

버블 정렬 과정

① 첫 번째 원소부터 인접한 원소끼리 계속 자리를 교환하면서 맨 마지막 자리까지 이동한다.

② 한 단계가 끝나면 가장 큰 원소가 마지막 자리로 정렬된다.

③ 교환하며 자리를 이동하는 모습이 물 위에 올라오는 거품 모양과 같다고 하여 버블 정렬이라고 한다.

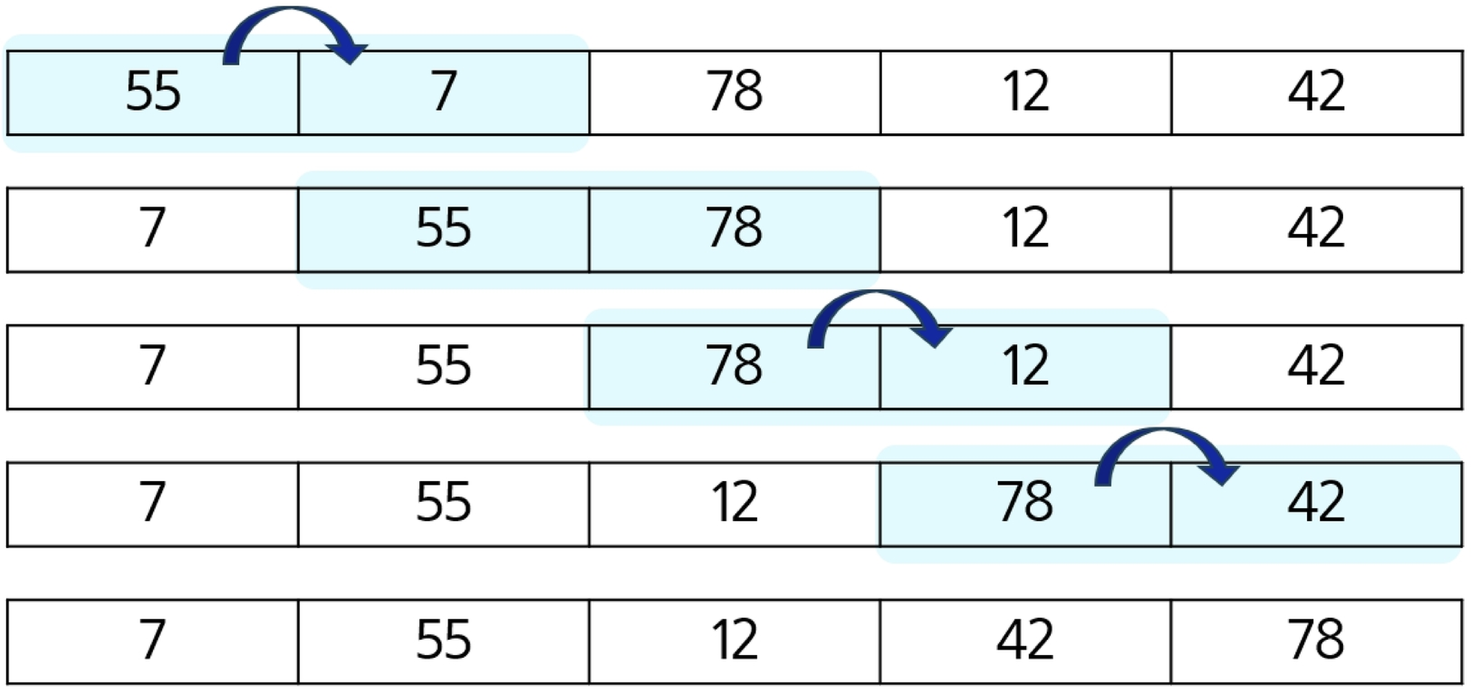
시간 복잡도

- $O(n^2)$

버블 정렬 과정 예시

- [55, 7, 78, 12, 42]를 버블 정렬하는 과정 (오름차순)

첫 번째 패스

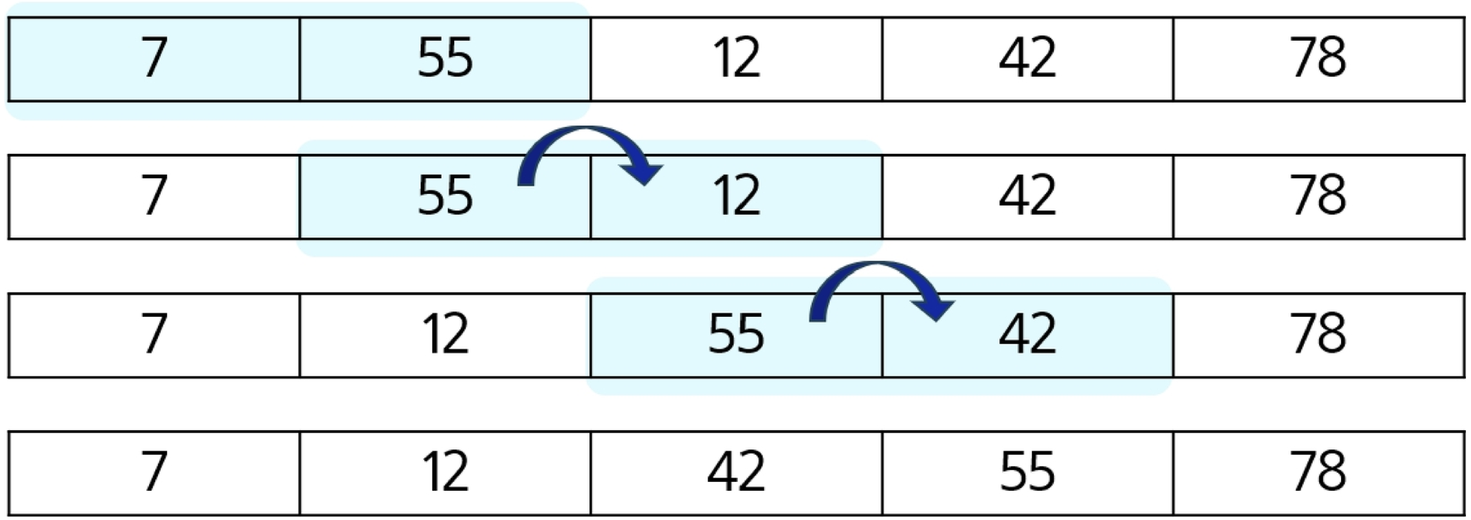


<그림13_SW문제해결 기본_List1_버블 정렬 첫 번째 패스>

버블 정렬 과정 예시

- [55, 7, 78, 12, 42]를 버블 정렬하는 과정 (오름차순)

두 번째 패스



<그림14_SW문제해결 기본_List1_버블 정렬 두 번째 패스>

버블 정렬 과정 예시

- [55, 7, 78, 12, 42]를 버블 정렬하는 과정 (오름차순)

세 번째 패스

7	12	42	55	78
7	12	42	55	78
7	12	42	55	78

<그림15_SW문제해결 기본_List1_버블 정렬 세 번째 패스>

버블 정렬 과정 예시

- [55, 7, 78, 12, 42]를 버블 정렬하는 과정 (오름차순)

네 번째 패스

7	12	42	55	78

7	12	42	55	78

<그림16_SW문제해결 기본_List1_버블 정렬 네 번째 패스>

정렬 끝

7	12	42	55	78

<그림17_SW문제해결 기본_List1_버블 정렬 완료>

버블 정렬 알고리즘

- 배열을 활용한 버블 정렬 알고리즘 예시(오름차순)

앞서 살펴 본 정렬 과정을 의사코드(pseudocode)로 구현하면 아래와 같습니다

```
BubbleSort(a, N)                        # 정렬할 배열과 배열의 크기
    for i : N-1 -> 1                    # 정렬할 구간의 끝
        for j : 0 -> i-1                # 비교할 원소 중 왼쪽 원소의 인덱스
            if a[j] > a[j+1]            # 왼쪽 원소가 더 크면
                a[j] <-> a[j+1]         # 오른쪽 원소와 교환
```

버블 정렬 알고리즘

- 배열을 활용한 버블 정렬 알고리즘 예시(오름차순)

앞서 살펴 본 정렬 과정을 코드로 구현하면 아래와 같습니다.

```
def bubble_sort(a, N) :          # 정렬할 List, N 원소 수
    for i in range(N-1, 0, -1) : # 범위의 끝 위치
        for j in range(i) :              # 비교할 왼쪽 원소 인덱스 j
            if a[j] > a[j+1] :
                a[j], a[j+1] = a[j+1], a[j]
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

카운팅 정렬

카운팅 정렬의 정의

카운팅 정렬
Counting Sort

항목들의 순서를 결정하기 위해
집합에 각 항목이 몇 개씩 있는지 세는 작업을 하여,
선형 시간에 정렬하는 효율적인 방식

카운팅 정렬 제한 사항

① 정수나 정수로 표현할 수 있는 자료에 대해서만 적용 가능
- 각 항목의 발생 횟수를 기록하기 위해, 정수 항목으로 인덱스 되는 카운트들의 배열을 사용하기 때문이다.

② 카운트들을 위한 충분한 공간을 할당하려면 집합 내의 가장 큰 정수를 알아야 함

시간 복잡도

- O(n + k) : n은 리스트 길이, k는 정수의 최댓값

카운팅 정렬 과정 예시

- [0, 4, 1, 3, 1, 2, 4, 1] 을 카운팅 정렬하는 과정

1단계 : DATA에서 각 항목들의 발생 횟수를 세고, 정수 항목들로 직접 인덱스 되는 카운트 배열 COUNTS에 저장한다.

<그림18_SW문제해결 기본_List1_카운팅 정렬 1단계>

카운팅 정렬 과정 예시

2단계 : 정렬된 집합에서 각 항목의 앞에 위치할 항목의 개수를 반영하기 위해 COUNTS의 원소를 조정한다.

INDEX	0	1	2	3	4
COUNTS	1	3	1	1	2

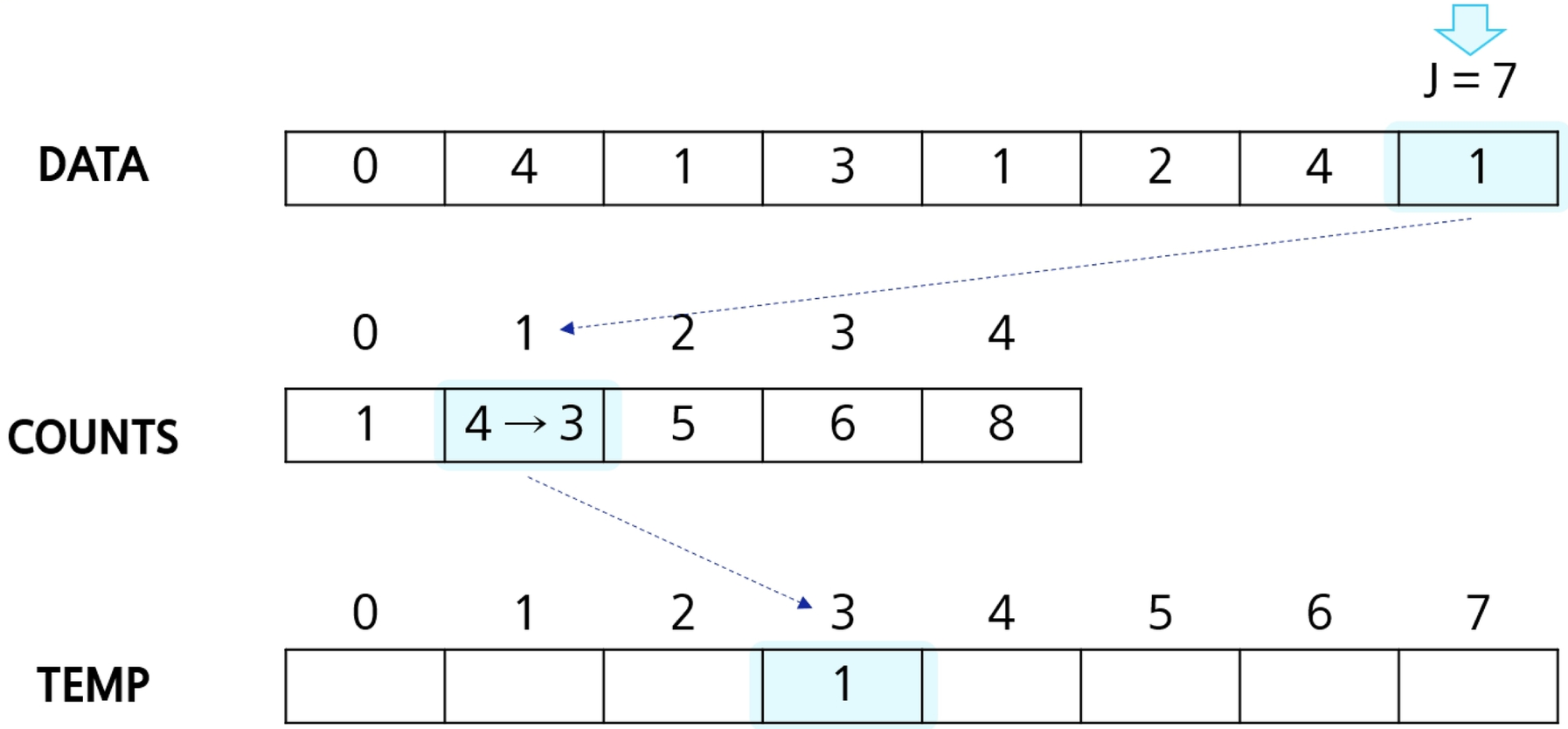
	1	4	5	6	8

COUNTS[i] = COUNTS[i-1] + COUNTS[i]

<그림19_SW문제해결 기본_List1_카운팅 정렬 2단계>

카운팅 정렬 과정 예시

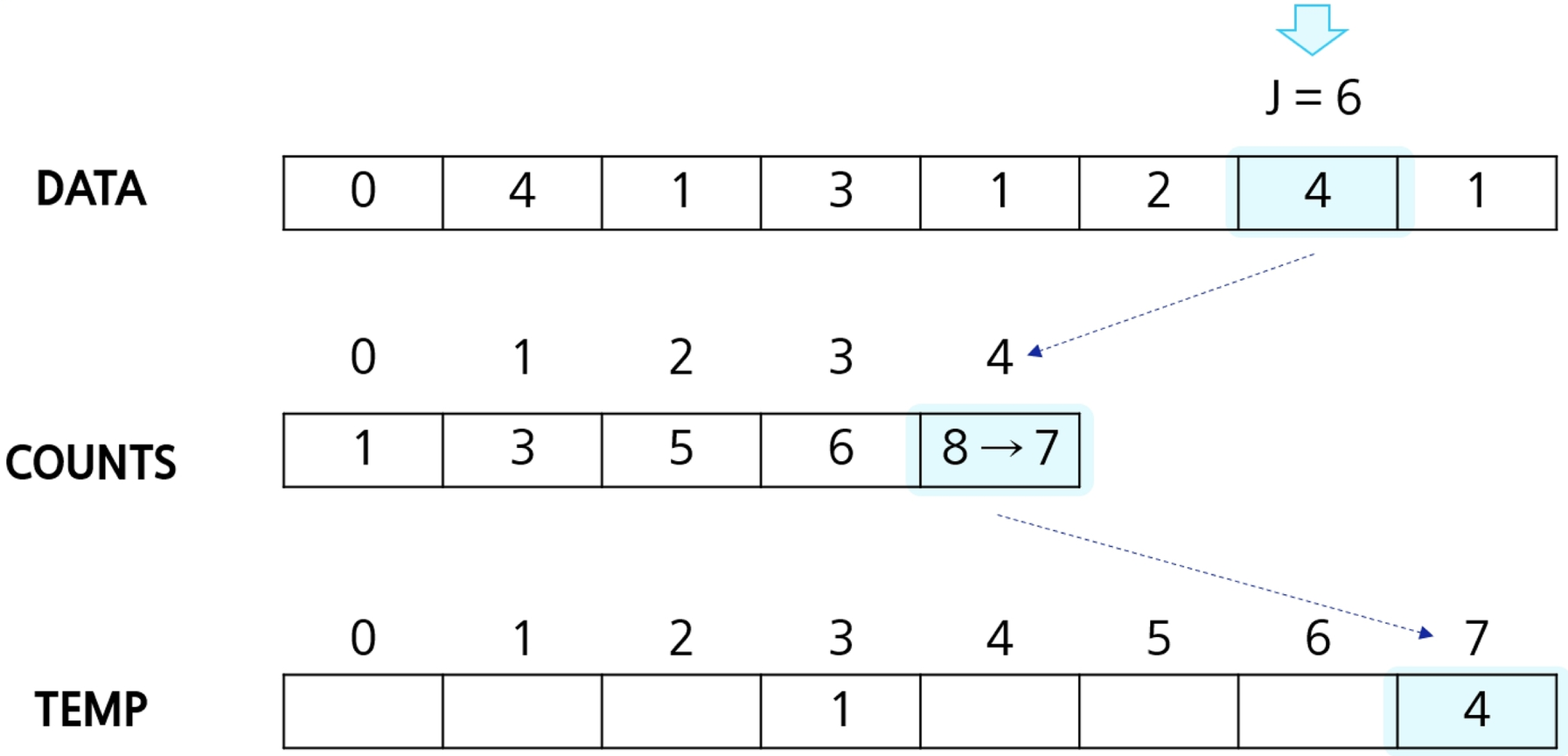
3단계 : DATA의 마지막 원소 1의 발생 횟수 COUNTS[1]을 감소시키고 TEMP에 1을 삽입한다.



<그림20_SW문제해결 기본_List1_카운팅 정렬 3단계>

카운팅 정렬 과정 예시

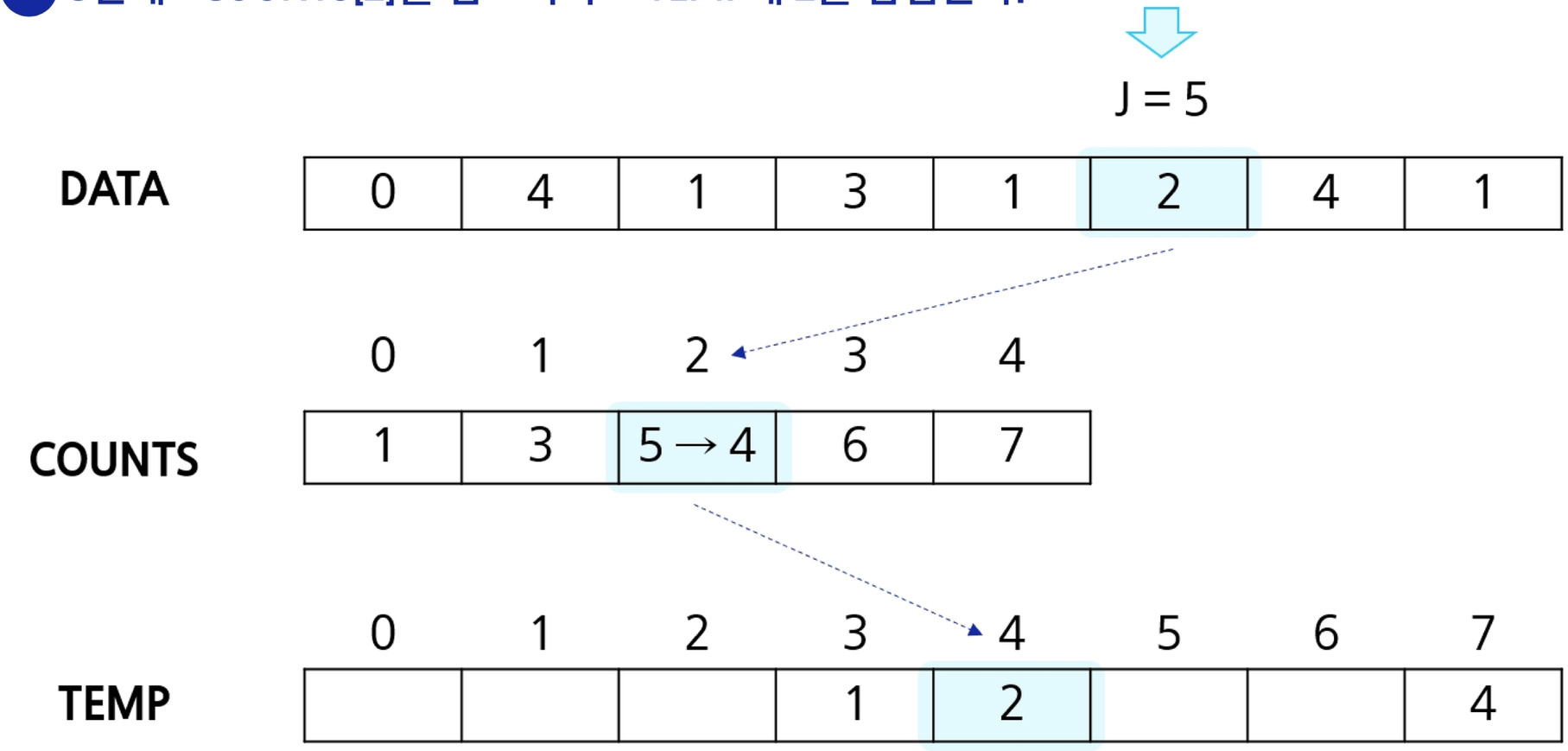
3단계 : COUNTS[4]를 감소시키고 TEMP에 4를 삽입한다.



<그림21_SW문제해결 기본_List1_카운팅 정렬 3단계 반복>

카운팅 정렬 과정 예시

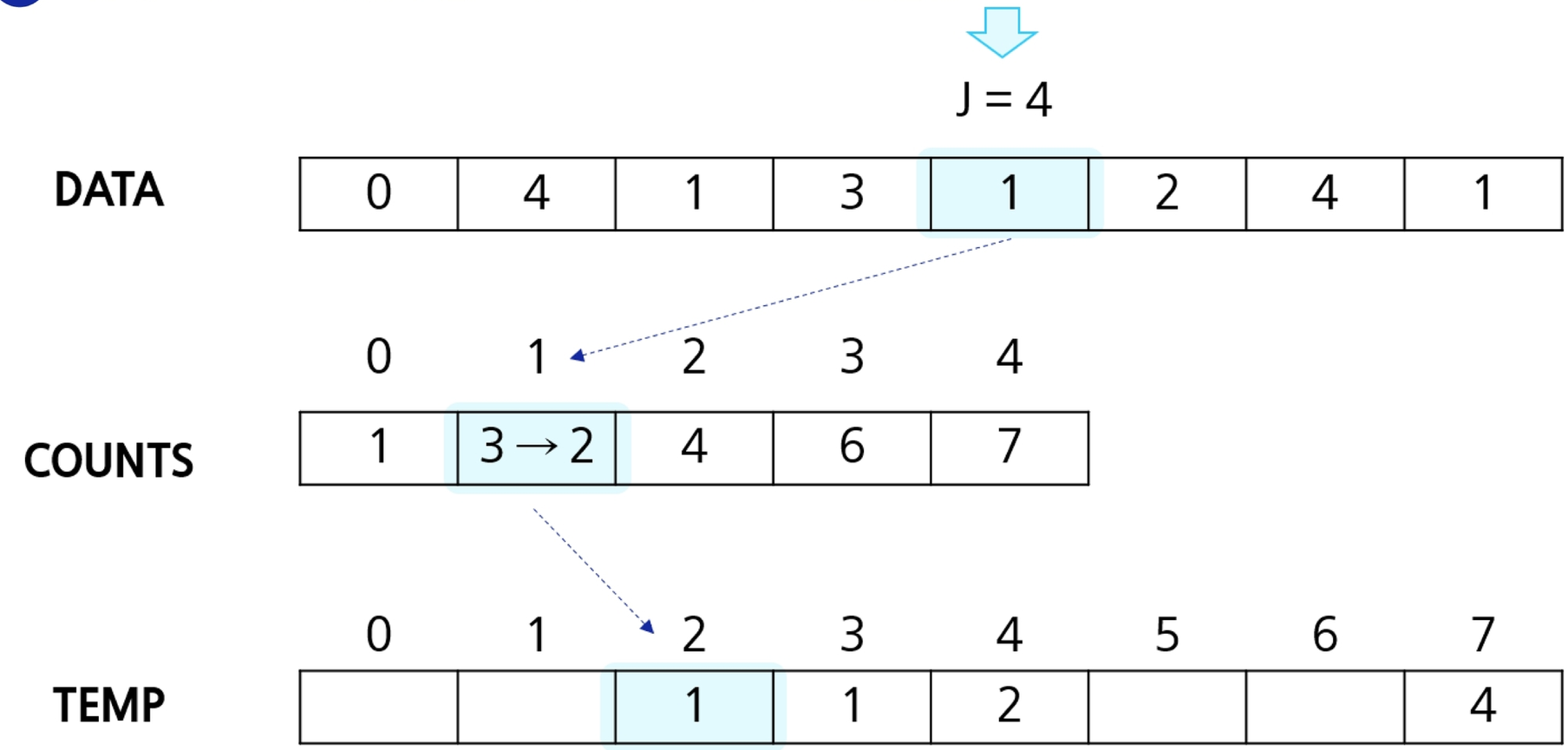
3단계 : COUNTS[2]를 감소시키고 TEMP에 2를 삽입한다.



<그림22_SW문제해결 기본_List1_카운팅 정렬 3단계 반복>

카운팅 정렬 과정 예시

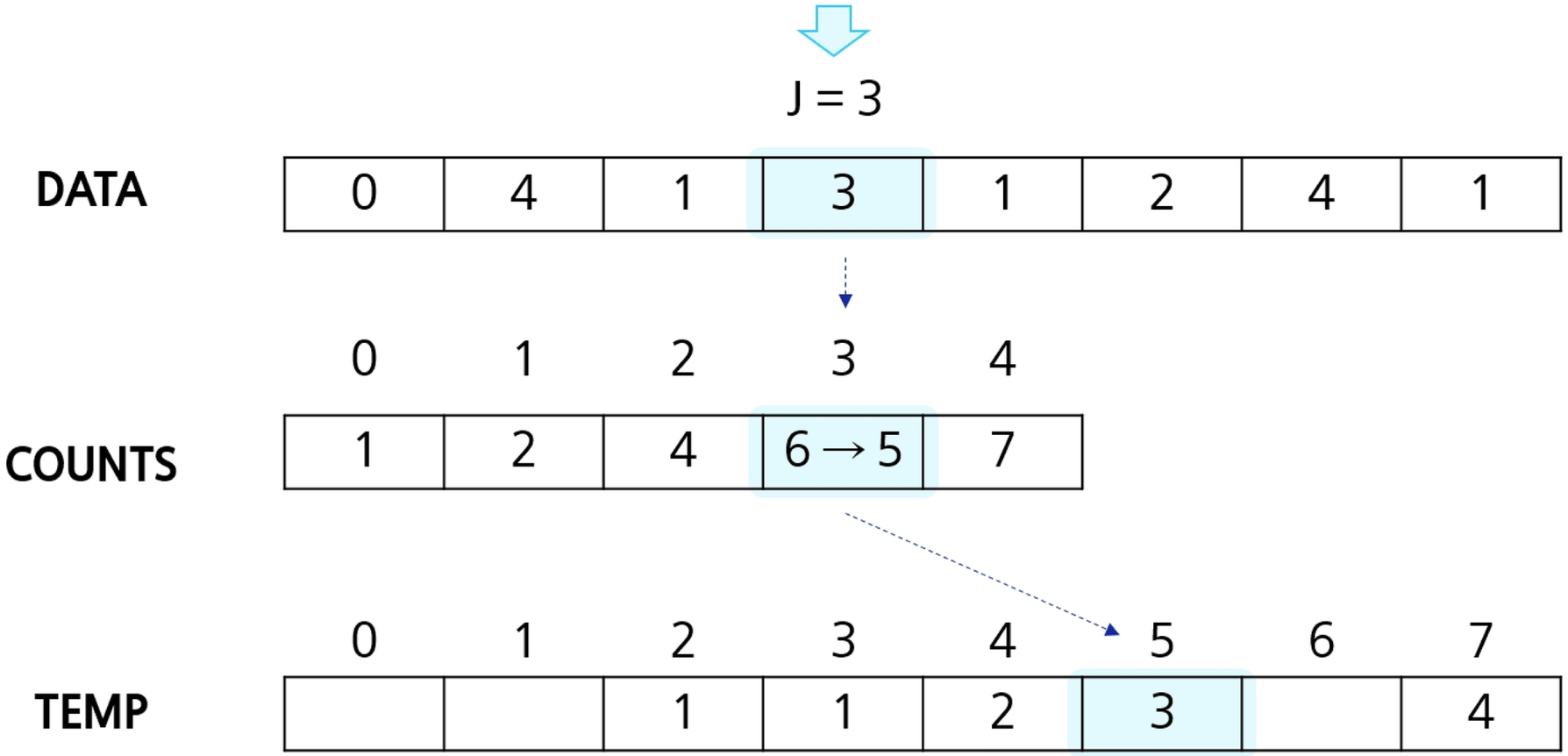
3단계 : COUNTS[1]을 감소시키고 TEMP에 1을 삽입한다.



<그림23_SW문제해결 기본_List1_카운팅 정렬 3단계 반복>

카운팅 정렬 과정 예시

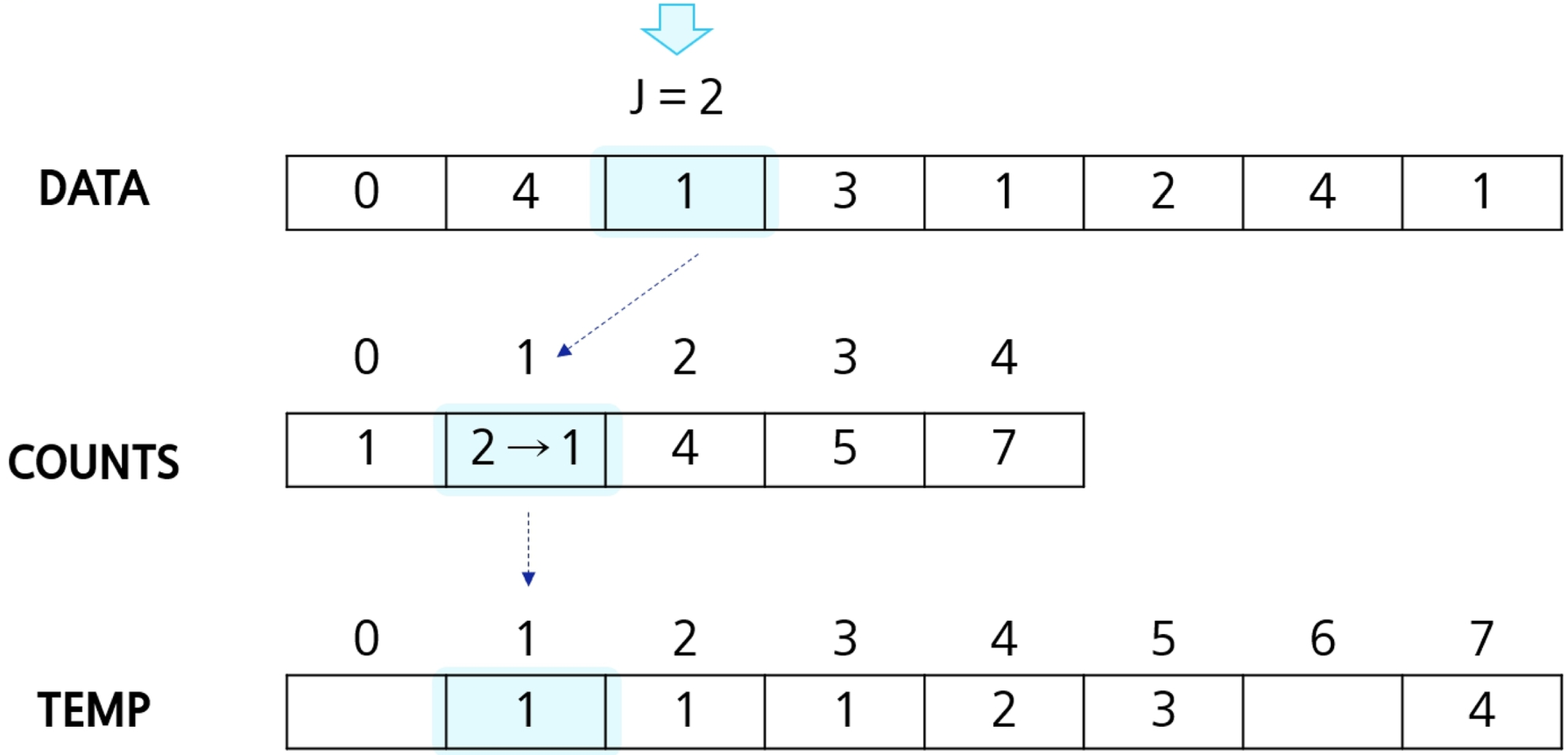
3단계 : COUNTS[3]을 감소시키고 TEMP에 3을 삽입한다.



<그림24_SW문제해결 기본_List1_카운팅 정렬 3단계 반복>

카운팅 정렬 과정 예시

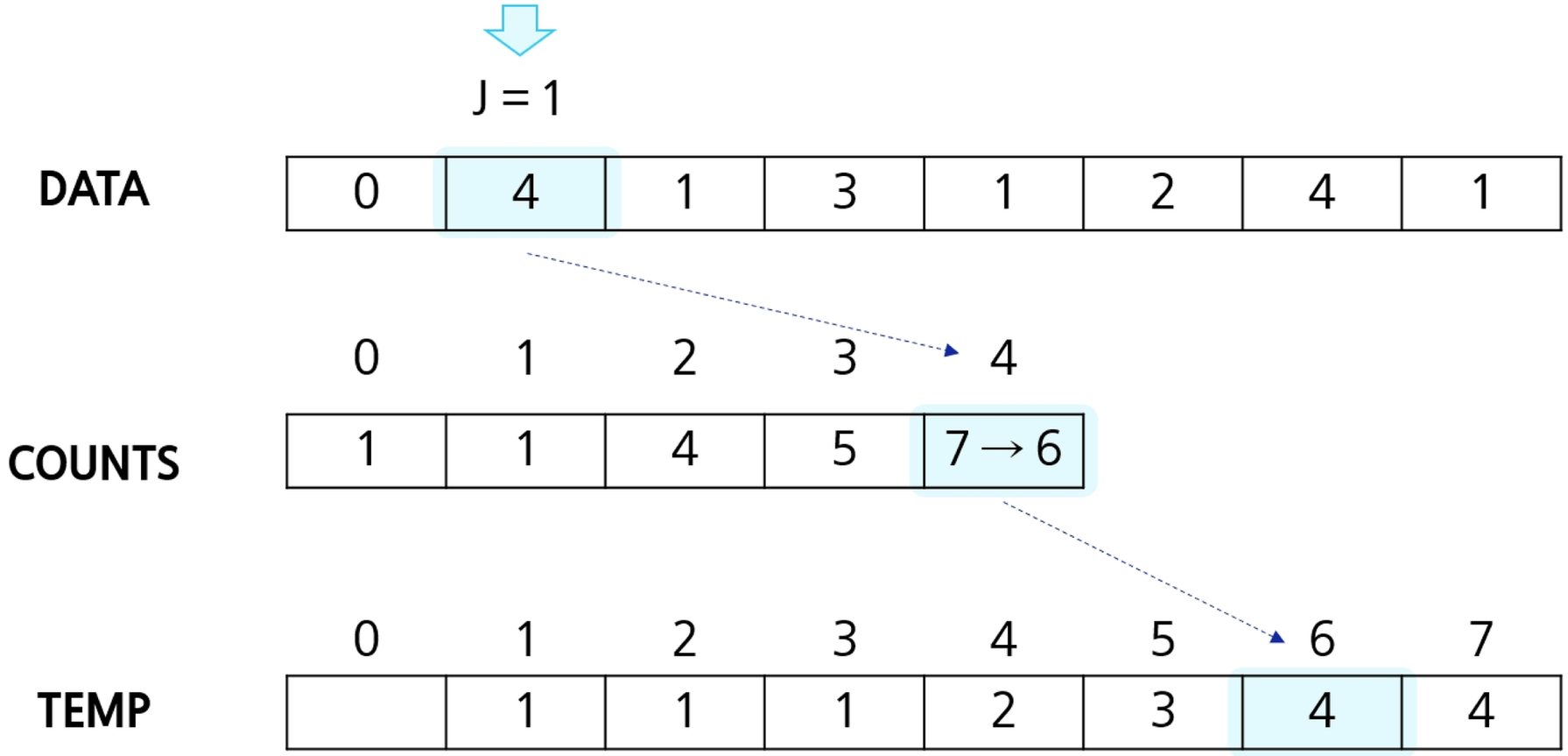
3단계 : COUNTS[1]을 감소시키고 TEMP에 1을 삽입한다.



<그림25_SW문제해결 기본_List1_카운팅 정렬 3단계 반복>

카운팅 정렬 과정 예시

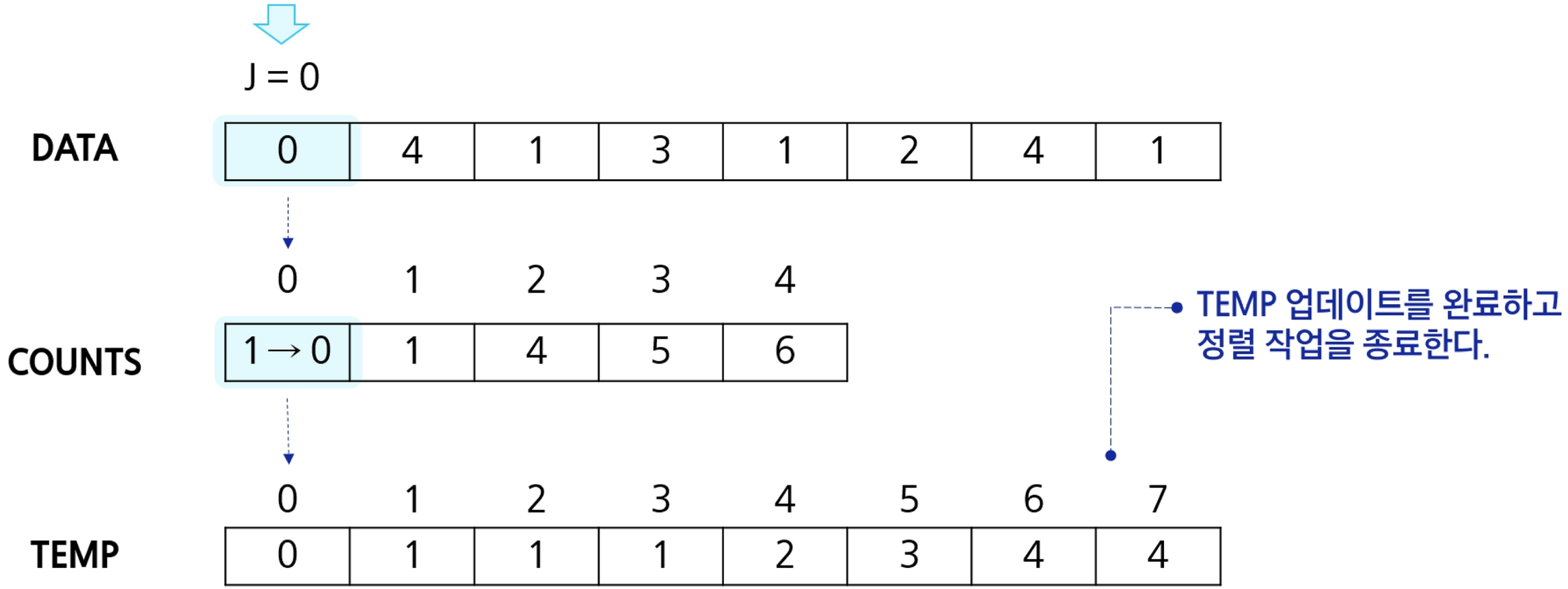
3단계 : COUNTS[4]를 감소시키고 TEMP에 4를 삽입한다.



<그림26_SW문제해결 기본_List1_카운팅 정렬 3단계 반복>

카운팅 정렬 과정 예시

3단계 : COUNTS[0]를 감소시키고 TEMP에 0을 삽입한다.



<그림27_SW문제해결 기본_List1_카운팅 정렬 3단계 반복>

카운팅 정렬 알고리즘

앞서 살펴 본 정렬 과정을 코드로 구현하면 아래와 같습니다.

```
def counting_sort(DATA, TEMP, k):
    # DATA [] -- 입력 배열(원소는 0 이상 k이하 정수)
    # TEMP [] -- 정렬된 배열
    # COUNTS [] -- 카운트 배열

    COUNTS = [0] * (k+1)

    for i in range (len(DATA)) : # DATA[i] 발생횟수 기록
        COUNTS[DATA[i]] += 1

    for i in range (1, k+1) :    # COUNTS 값 조정 (누적)
        COUNTS[i] += COUNTS[i-1]

    for i in range (len(DATA)-1, -1, -1) : # 3단계
        COUNTS[DATA[i]] -= 1
        TEMP[COUNTS[DATA[i]]] = DATA[i]
```

정렬 알고리즘 비교

알고리즘	평균 수행시간	최악 수행시간	알고리즘 기법	비고
버블 정렬	$O(n^2)$	$O(n^2)$	비교와 교환	코딩이 가장 손쉽다.
카운팅 정렬	O(n+k)	O(n+k)	비교환 방식	k가 비교적 작을 때만 가능하다.
선택 정렬	$O(n^2)$	$O(n^2)$	비교와 교환	교환의 회수가 버블, 삽입정렬보다 작다.
퀵 정렬	O(n log n)	$O(n^2)$	분할 정복	최악의 경우 $O(n^2)$ 이지만, 평균적으로는 가장 빠르다.
삽입 정렬	$O(n^2)$	$O(n^2)$	비교와 교환	n의 개수가 작을 때 효과적이다.
병합 정렬	O(n log n)	O(n log n)	분할 정복	연결리스트의 경우 가장 효율적인 방식이다.

<표4_SW문제해결 기본_List1_정렬 알고리즘 비교>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

완전 검색

Baby-gin Game

문제 설명

0~9 사이의 숫자 카드에서 임의의 카드 6장을 뽑았을 때, 3장의 카드가 연속적인 번호를 갖는 경우를 run이라 하고, 3장의 카드가 동일한 번호를 갖는 경우를 triplet이라고 한다.

그리고, 6장의 카드가 run과 triplet로만 구성된 경우를 baby-gin으로 부른다.

6자리의 숫자를 입력 받아 baby-gin 여부를 판단하는 프로그램을 작성하라.

Baby-gin Game

입력 예시

- 667767은 두 개의 triplet이므로 baby-gin이다. (666, 777)
- 054060은 한 개의 run과 한 개의 triplet이므로 역시 baby-gin이다. (456, 000)
- 101123은 한 개의 triplet가 존재하나, 023이 run이 아니므로 baby-gin 이 아니다.
  (123을 run으로 사용하더라도 011이 run이나 triplet가 아님)

Q 6자리의 숫자를 입력 받아 어떻게 Baby-gin 여부를 찾을 것인가?

완점 검색의 정의

완전 검색
Exhaustive Search

문제의 해법으로 생각할 수 있는
모든 경우의 수를 나열해 보고 확인하는 기법

Brute-force 혹은 generate-and-test 기법이라고도 불립니다.
모든 경우의 수를 테스트한 후, 최종 해법을 도출하는 방법으로 일반적으로
경우의 수가 상대적으로 작을 때 유용합니다.

완전 검색의 필요성

- 완전 검색으로 시작

 - 모든 경우의 수를 생성하고 테스트하기 때문에 수행 속도는 느리지만, 해답을 찾아내지 못할 확률이 작습니다.

 - 자격검정평가 등에서 주어진 문제를 풀 때, 우선 완전 검색으로 접근하여 해답을 도출한 후, 성능 개선을 위해 다른 알고리즘을 사용하고 해답을 확인하는 것이 바람직합니다.

완전 검색을 활용한 Baby-gin 접근

고려할 수 있는 모든 경우의 수 생성하기

- 6개의 숫자로 만들 수 있는 모든 숫자 나열 (중복 포함)
- 예) 입력으로 [2, 3, 5, 7, 7, 7]을 받았을 경우, 아래와 같이 순열을 생성할 수 있습니다.

2	3	5	7	7	7
2	3	7	5	7	7
2	3	7	7	5	7
		…			
7	7	7	5	3	2

완전 검색을 활용한 Baby-gin 접근

해답 테스트하기

- 앞의 3자리와 뒤의 3자리를 잘라, run과 triplet 여부를 테스트하고 최종적으로 baby-gin을 판단합니다.

<그림28_SW문제해결 기본_List1_baby-gin 해답 테스트>

순열 (Permutation)

- 서로 다른 것들 중 몇 개를 뽑아서 한 줄로 나열하는 것입니다.
- 서로 다른 n개 중 r개를 택하는 순열은 아래와 같이 표현합니다.

$$nPr$$

- 그리고 nPr은 다음과 같은 식이 성립합니다.

$$nPr = n * (n-1) * (n-2) * \cdots * (n-r+1)$$

- nPn = n!이라고 표기하며 Factorial이라 부릅니다.

$$n! = n * (n-1) * (n-2) * \cdots * 2 * 1$$

단순하게 순열을 생성하는 방법

- {1, 2, 3}을 포함하는 모든 순열을 생성하는 함수
  - 동일한 숫자가 포함되지 않았을 때, 각 자릿수별로 loop을 이용해 아래와 같이 구현할 수 있습니다.

```
for i1 in range(1, 4):
    for i2 in range(1, 4):
        if i2 != i1 :
            for i3 in range(1, 4):
                if i3 != i1 and i3 != i2 :
                    print(i1, i2, i3)
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

탐욕 알고리즘

탐욕 알고리즘의 정의

탐욕 알고리즘
Greedy

여러 경우 중 하나를 결정해야 할 때마다
그 순간에 최적이라고 생각되는 것을 선택해 나가는 방식으로 진행하여
최종적인 해답에 도달하는 방식

탐욕 알고리즘 특징

- 최적해를 구하는 데 사용되는 근시안적인 방법
- 각 선택의 시점에서 이루어지는 결정은 지역적으로는 최적이지만, 그 선택들을 계속 수집하여 최종적인 해답을 만들었다고 하여, 그것이 최적이라는 보장은 없음
- 일반적으로, 머릿속에 떠오르는 생각을 검증 없이 바로 구현하면 Greedy 접근임

탐욕 알고리즘 과정

① 해 선택 : 현재 상태에서 부분 문제의 최적 해를 구한 뒤, 이를 부분 해 집합(Solution Set)에 추가한다.

② 실행 가능성 검사 : 새로운 부분 해 집합이 실행 가능한지를 확인한다. 곧, 문제의 제약 조건을 위반하지 않는지를 검사한다.

③ 해 검사 : 새로운 부분 해 집합이 문제의 해가 되는지를 확인한다. 아직 전체 문제의 해가 완성되지 않았다면 ①의 해 선택부터 다시 시작한다.

탐욕 알고리즘 예시

- 거스름돈 줄이기

"어떻게 하면 손님에게 거스름돈으로 주는
지폐와 동전의 개수를 최소한으로 줄일 수 있을까?"

1단계 해 선택

- 여기에서는 멀리 내다볼 것 없이 가장 좋은 해를 선택합니다.
- 단위가 큰 동전으로만 거스름돈을 만들면 동전의 개수가 줄어들므로 현재 고를 수 있는 가장 단위가 큰 동전을 하나 골라 거스름돈에 추가합니다.

탐욕 알고리즘 예시

- 거스름돈 줄이기

2단계 실행 가능성 검사

- 거스름돈이 손님에게 내드려야 할 액수를 초과하는지 확인합니다.
- 초과한다면 마지막에 추가한 동전을 거스름돈에서 빼고, 1단계로 돌아가서 현재보다 한 단계 작은 단위의 동전을 추가합니다.

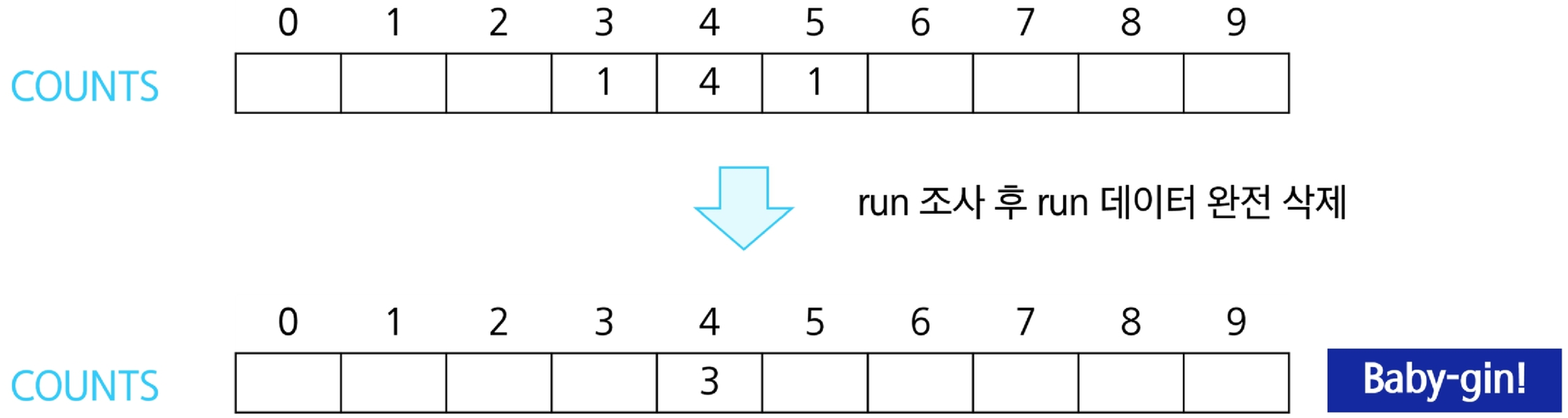
3단계 해 선택

- 거스름돈 문제의 해는 당연히 거스름돈이 손님에게 내드려야 하는 액수와 일치합니다. 더 드려도, 덜 드려도 안되기 때문에 거스름돈을 확인해서 액수에 모자라면 다시 1단계로 돌아가서 거스름돈에 추가할 동전을 고릅니다.

탐욕 알고리즘을 활용한 Baby-gin 접근

- 6개의 숫자는 6자리의 정수 값으로 입력된다.
- counts 배열의 각 원소를 체크하여 run과 triplet 및 baby-gin 여부를 판단한다.

입력 예시 : 444345



<그림29_SW문제해결 기본_List1_탐욕 알고리즘 baby-gin>

입력 예시 : 333456

	0	1	2	3	4	5	6	7	8	9
COUNTS				3	1	1	1			

run 조사 후 run 데이터 완전 삭제?

	0	1	2	3	4	5	6	7	8	9
COUNTS				2	0	0	1			

<그림30_SW문제해결 기본_List1_탐욕 알고리즘 baby-gin 오류>

입력 예시 : 444456

	0	1	2	3	4	5	6	7	8	9
COUNTS					4	1	1			

triplet 조사 후 triplet 데이터 완전 삭제

	0	1	2	3	4	5	6	7	8	9
COUNTS					1	1	1			

Baby-gin!

<그림31_SW문제해결 기본_List1_탐욕 알고리즘 baby-gin 정상>

탐욕 알고리즘을 활용한 Baby-gin 접근

앞서 살펴 본 풀이를 코드로 구현하면 아래와 같습니다.

```
num = 456789 # Baby Gin 확인할 6자리 수
c = [0] * 12 # 6자리 수로부터 각 자리 수를 추출하여 개수를 누적할 리스트

for i in range(6) :
    c[num % 10] += 1
    num //= 10
```

탐욕 알고리즘을 활용한 Baby-gin 접근

```
i = 0
tri = run = 0
while i < 10 :
    if c[i] >= 3 : # triplete 조사 후 데이터 삭제
        c[i] -= 3
        tri += 1
        continue
    if c[i] >= 1 and c[i+1] >= 1 and c[i+2] >= 1 : # run 조사 후 데이터 삭제
        c[i] -= 1
        c[i+1] -= 1
        c[i+2] -= 1
        run += 1
        continue
    i += 1

if run + tri == 2 : print("Baby Gin")
else : print("Lose")
```

Baby-gin 문제에서 자주 실수하는 오답

- 입력받은 숫자를 정렬한 후, 앞뒤 3자리씩 끊어서 run 및 triplet을 확인하는 방법을 고려할 수도 있다.

  예) [6, 4, 4, 5, 4, 4]
  정렬하여 [4, 4, 4, 4, 5, 6]을 얻어내면 쉽게 baby-gin을 확인할 수 있다.

  예) [1, 2, 3, 1, 2, 3]
  정렬하면 [1, 1, 2, 2, 3, 3]로서, 오히려 baby-gin 확인을 실패할 수 있다.

※ **위의 예처럼, 탐욕 알고리즘적인 접근은 해답을 찾아내지 못하는 경우도 있으니 유의해야 한다.**

실습

배열

- 4828. min max
- 4835. 구간합

카운트 배열

- 4834. 숫자 카드

탐욕기법

- 4831. 전기버스

추가 연습

- (User problem) 9386. 연속한 1의 개수
- 6485. 삼성시의 버스 노선
- 1966. 숫자를 정렬하자

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 최적성이 의미하는 것은?

a) 빠름
b) 간단함
c) 더 개선 불가
d) 적은 메모리

2 작업량을 판단하는 기준은?

a) 연산 횟수
b) 코드 길이
c) 변수 개수
d) 메모리

3 리스트의 길이를 구하는 함수는?

a) size()
b) length()
c) len()
d) count()

4 리스트의 첫 인덱스는?

a) 0
b) 1
c) -1
d) 2

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 O(n log n)보다 효율이 낮은 것은?

a) O(log n)
b) $O(n^2)$
c) O(1)
d) O(n)

6 arr =['a', 'b', 'c']일 때 arr[1] = 'x' 의 결과는?

a) ['a','b','c']
b) ['a','x','c']
c) ['x','b','c']
d) 에러

7 arr = [3, 1, 7, 4, 6, 7] 일 때,
max_idx 값으로 올바른 것은?

```
max_idx = 0
for i in range(1, 6):
    if arr[max_idx] <= arr[i]:
        max_idx = i
```

8 버블 정렬은 무엇을 비교하나요?

a) 첫값과 끝값
b) 인접한 값들
c) 가장 큰 값
d) 짝수 인덱스

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 [5, 3, 1]에서 첫 번째 버블 정렬 후 결과는?

a) [3, 1, 5]
b) [5, 1, 3]
c) [3, 5, 1]
d) [1, 3, 5]

10 카운팅 정렬은 어떤 방식인가요?

a) 등장 횟수 세기
b) 위치 바꾸기
c) 나누기 정렬
d) 랜덤 정렬

11 카운팅 정렬의 전제 조건은?

a) 정렬된 데이터
b) 값이 정수
c) 문자열
d) 리스트 길이 고정

12 완전 검색은 어떤 방법인가요?

a) 무작위 선택
b) 빠르게 추정
c) 가능한 경우 모두 확인
d) 하나만 확인

확인문제

정답 및 해설

1. c) 더 개선 불가 2. a) 연산 횟수 3. c) len() 4. a) 0 5. b) $O(n^2)$ 6. b) ['a','x','c'] 7. 5
8. b) 인접한 값들 9. b) [3, 1, 5] 10. a) 등장 횟수 세기 11. b) 값이 정수 12. c) 가능한 경우 모두 확인

1. 최적성은 더 이상 개선할 수 없는 상태를 의미합니다.

2. 작업량은 연산 횟수로 판단합니다.

3. 리스트 길이는 len()으로 구합니다.

4. 파이썬 리스트는 0부터 시작합니다.

5. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$

6. 인덱스 1의 값이 'x'로 바뀝니다.

7. 리스트에서 최댓값이 여러 개인 경우 마지막 인덱스 max_idx를 찾는 코드입니다. 따라서 max_idx = 5 입니다.

8. 버블 정렬은 인접한 값들을 비교합니다.

9. 첫 번째 버블 정렬 과정은 아래와 같습니다.
첫 번째 비교: 5와 3 → 큰 수인 5가 뒤로 가야 하므로 자리를 바꿈 → [3, 5, 1]
두 번째 비교: 5와 1 → 다시 큰 수 5가 뒤로 가야 하므로 바꿈 → [3, 1, 5]

10. 카운팅 정렬은 등장 횟수를 세어 리스트를 정렬하는 방법입니다.

11. 카운팅 정렬은 정수 값에 적합합니다.

12. 완전 탐색은 가능한 경우를 모두 확인합니다.

활동정리

핵심키워드

개념	설명
알고리즘(Algorithm)	유한한 단계를 통해 문제를 해결하기 위한 절차나 방법
배열(Array)	일정한 자료형의 변수들을 하나의 이름으로 열거하여 사용하는 자료구조
정렬(Sort)	2개 이상의 자료를 작은 값부터 큰 값 혹은 그 반대의 순서대로 재배열하는 것
버블 정렬(Bubble Sort)	인접한 두 개의 원소를 비교하며 자리를 계속 교환하며 정렬하는 알고리즘
카운팅 정렬(Counting Sort)	집합에 각 항목이 몇 개씩 있는지 세는 작업을 하여 정렬하는 알고리즘
완전 검색(Exaustive Search)	문제의 해법으로 생각할 수 있는 모든 경우의 수를 나열해보고 확인하는 기법
탐욕(Greedy) 알고리즘	여러 경우 중 하나를 결정해야 할 때마다 그 순간에 최적이라고 생각되는 것을 선택해 나가는 방식으로 진행하여 최종적인 해답에 도달하는 알고리즘

<표5_SW문제해결 기본_List1_핵심키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **알고리즘이란?**
  - 유한한 단계를 통해 문제를 해결하기 위한 절차나 방법
  - 시간복잡도를 통해 알고리즘의 성능을 판단

- **배열이란?**
  - 일정한 자료형의 변수들을 하나의 이름으로 열거하여 사용하는 자료구조
  - 배열을 사용하면 하나의 선언을 통해서 둘 이상의 변수를 선언할 수 있다

활동정리

오늘 공부한 내용 요약 및 정리

- **정렬이란?**
  - 2개 이상의 자료를 특정 기준에 의해 작은 값부터 큰 값 혹은 그 반대의 순서대로 재배열하는 것
  - 버블 정렬 : 인접한 두 개의 원소를 비교하며 자리를 계속 교환하며 정렬하는 방식
  - 카운팅 정렬 : 집합에 각 항목이 몇 개씩 있는지 세는 작업을 하여 정렬하는 방식

- **완전 검색이란?**
  - 문제의 해법으로 생각할 수 있는 모든 경우의 수를 나열해 보고 확인하는 기법
  - 일반적으로 경우의 수가 상대적으로 작을 때 유용

- **탐욕 알고리즘이란?**
  - 여러 경우 중 하나를 결정해야 할 때마다 그 순간에 최적이라고 생각되는 것을 선택해 나가는 방식으로 진행하여 최종적인 해답에 도달

활동정리

내가 좋아하는 음식 리스트를 만들어 봅시다!

1. 변수 [ ]에 내가 좋아하는 음식들을 리스트 형태로 저장합니다
2. 리스트에 몇 개의 음식이 들어 있는지 확인하기 위해 [ ]을 사용해 봅니다
3. 리스트에서 세 번째 음식을 꺼내기 위해 [ ] 형태로 특정 값을 출력합니다

활동정리

내가 좋아하는 음식 리스트를 만들어 봅시다!

1. 변수 foods에 내가 좋아하는 음식들을 리스트 형태로 저장합니다
2. 리스트에 몇 개의 음식이 들어 있는지 확인하기 위해 len(foods)을 사용해 봅니다
3. 리스트에서 세 번째 음식을 꺼내기 위해 foods[2] 형태로 특정 값을 출력합니다

```
foods = ['apple', 'banana', 'orange', 'peach']
print(len(foods))
print(foods[2])
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다